Evaluation Validity in Software-Defined Network Intrusion Detection: An Empirical Audit of Controller Observability and Host Generalization on InSDN

Adhyan Jain*
Vellore Institute of Technology
Vellore, India
adhyan.jain2024@vitstudent.ac.in

Atharva Sheersh Pandey*
Vellore Institute of Technology
Vellore, India
atharva.sheersh2024@vitstudent.ac.in

Kshitij Manikshete*
Vellore Institute of Technology
Vellore, India
kshitij.rahul2024@vitstudent.ac.in

***Abstract*—Machine-learning intrusion detection for Software-Defined Networks (SDN) is usually evaluated on flow-record benchmarks computed offline at end hosts. We audit the validity of this practice on InSDN, the public SDN benchmark that also ships the OpenFlow control-channel captures of its ONOS controller, with CICIDS2017 as an external reference. Decoding all 44 control-channel captures (7 381 679 messages) shows that no flow-table entry or FlowMod matches IP addresses or ports, so per-flow counters are absent; ONOS instead punts IPv4 packets to the controller (an IPv4 punt rule is present in 44 of 44 captures), and the controller receives 0.68–1.29 PacketIns per target-side packet in 9 of 12 attack scenarios. Flows reassembled from PacketIns reproduce packet totals on probing traffic but only 38% and 45% of the host-side totals on DoS and benign traffic. A controller-side MLP reaches 0.946 balanced accuracy on unseen captures, but InSDN records benign and attack traffic in separate sessions: an "any IP PacketIn" rule scores 0.995 within attack sessions, a host-identity rule learned on training captures alone reaches 0.734, and the MLP's features separate sessions at 0.940 after attack packets are removed, so the score cannot be read as attack detection. Under strict labelling, OVS DDoS and OVS brute-force contribute no attack windows and probing supplies 76% of the rest. On flow features, random splits give 0.9998 (InSDN) and 0.9917 (CICIDS2017) balanced accuracy, whereas holding out source hosts gives 0.907 ± 0.085 (worst split 0.66) and 0.641 ± 0.056; the InSDN dispersion is consistent with which of eight dominant hosts is withheld. A targeted audit of 12 SDN-IDS papers finds none that reports a host-grouped split. All findings concern one SDN dataset and one controller. We propose seven evaluation guidelines and release the code and result files behind every reported number.**

***Index Terms*—Software-defined networking, intrusion detection systems, OpenFlow, ONOS, InSDN, control-plane telemetry, evaluation methodology, data leakage, generalization.**

I. Introduction

Software-Defined Networking (SDN) decouples network control logic from underlying forwarding hardware, enabling centralized programmability and dynamic flow management [1], [2]. Motivated by the controller's global architectural view, a substantial literature proposes machine learning (ML) and deep learning Network Intrusion Detection Systems (NIDS) designed to secure SDN environments [3], [4], [5].

*All three authors contributed equally to this work.

Despite widespread interest, a methodological gap separates published evaluations from operational networking conditions. In practice, SDN intrusion detection research rarely interacts with an active SDN controller. Instead, researchers predominantly evaluate machine learning models on offline comma-separated value (CSV) benchmarks, such as InSDN [3] and CICIDS2017 [6]. These benchmark tables consist of bidirectional 5-tuple flow summaries computed by feature extractors like CICFlowMeter [7] operating directly on full-packet captures obtained from network taps at end hosts. Standard pipelines train high-capacity classifiers on 80+ statistical features—including inter-arrival time moments, bidirectional packet length distributions, and cumulative TCP flag counts—and consistently report detection accuracies exceeding 99.5% [8], [9], [10].

This prevalent methodology suffers from two foundational threats to validity, illustrated conceptually in Fig. 1:

1) **The Controller Observability Disconnect:** An OpenFlow controller does not have native access to packet-level timing moments or fine-grained per-flow payload metrics. Under standard OpenFlow 1.3 reactive forwarding [11], the controller receives `PacketIn` messages for initial flow misses, issues `FlowMod` entries, and periodically polls aggregate flow statistics counters. Whether the controller's installed flow table entries actually capture fine-grained 5-tuple counters depends on the controller's forwarding application. If the controller installs wildcard or MAC-pair rules, granular 5-tuple statistical features cannot be constructed at the controller without either altering flow-table granularity (risking TCAM exhaustion) or streaming full payloads (risking control-channel saturation).
2) **The Evaluation-Validity and Topology Leakage Disconnect:** In security machine learning, randomized train/test splits are known to suffer from severe data leakage when applied to benchmark datasets with repetitive traffic or homogenous host assignments [12], [13], [14], [15]. When flows from the same IP address or session appear in both training and test partitions, models learn

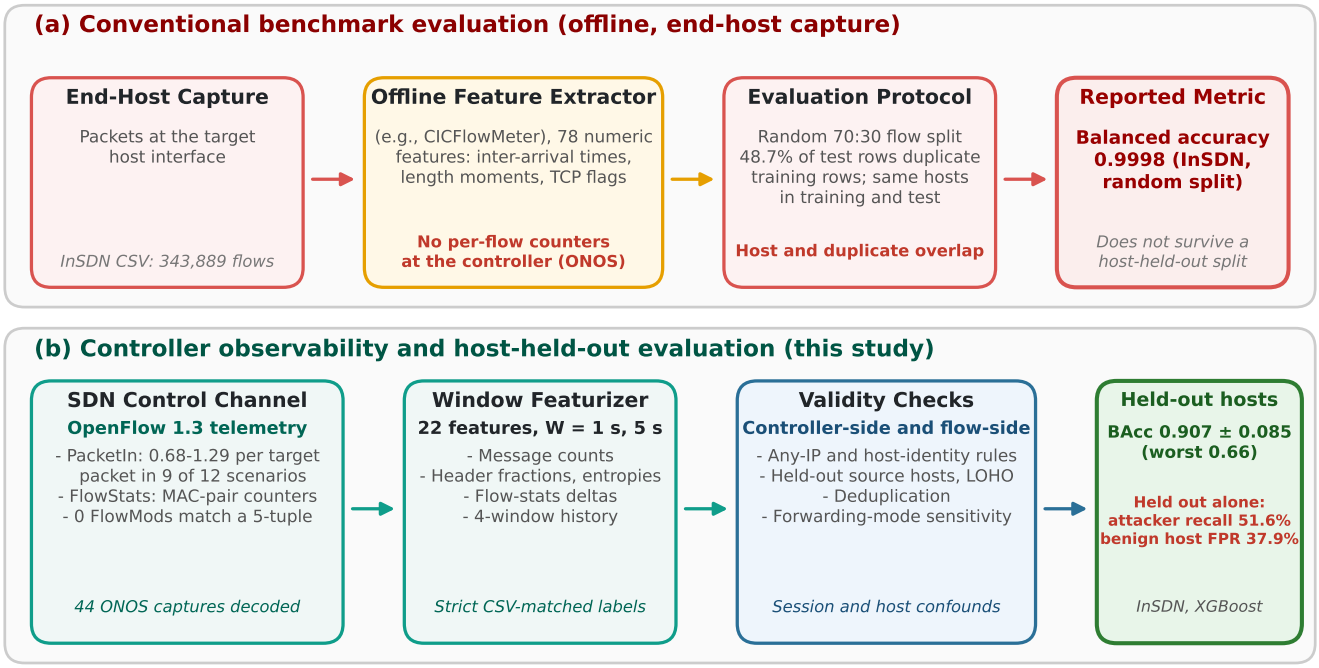


Fig. 1. Overview of the audit. (a) Conventional evaluation computes flow features offline from end-host captures and splits flows at random, with host overlap, 48.7% duplicate test rows and 0.9998 balanced accuracy on InSDN. (b) This study decodes the controller's OpenFlow 1.3 control channel (no flow-table entry or FlowMod matches a 5-tuple under ONOS's configuration), evaluates a controller-side detector against identity baselines, and repeats the flow-feature evaluation with source hosts held out, where balanced accuracy on InSDN is lower and far more variable.

spurious host-specific shortcut representations rather than generalizable attack signatures [16]. In SDN topologies with highly concentrated host structures, reported near-perfect accuracies may simply reflect the model's ability to recognize previously seen hosts rather than its capability to detect novel attackers.

While prior studies have highlighted feature calculation discrepancies [17], [18], [19], [20] or identified data smells in general non-SDN datasets [15], the specific manifestation of these issues within SDN control-plane environments remains largely unquantified. Crucially, the InSDN dataset release included not only offline flow CSVs but also the raw PCAP files of the ONOS controller's OpenFlow 1.3 control channel across all evaluated scenarios [3]. This provides a unique, empirical opportunity to examine what an SDN controller actually observes during benchmark execution and to systematically audit evaluation validity.

In this work we audit controller observability and evaluation validity for SDN intrusion detection on InSDN. We make five contributions, each a measurement and not a new detector:

1) **Control-channel characterization (§IV).** We decode all 44 ONOS OpenFlow 1.3 captures shipped with InSDN. No flow-statistics entry (24 437 MAC-pair and 80 464 `eth_type` observations) and none of the 3 827 FlowMods matches IP addresses or transport ports, so the labelled 5-tuple flows are not available as switch counters under this configuration. An IPv4-to-controller punt rule is present in all 44 captures, and the controller receives 0.68–1.29 PacketIns per target-side packet in 9 of 12 attack scenarios.
2) **What PacketIns can reconstruct (§IV).** A software flow reassembler with CICFlowMeter's expiration rules, run on one capture per scenario, recovers packet counts and durations in some scenarios and not in others, and its output is incomplete relative to host-side records (Table I).
3) **A validity analysis of controller-side detection (§V–§VII).** We show how label construction shapes the evaluated windows (Table II) and compare a controller-side MLP with an "any IP PacketIn" rule, host-identity rules learned on training captures, and a session probe. The 0.946 score cannot be taken as evidence of attack detection, because the benchmark confounds session identity with the label.
4) **A host-held-out audit (§VIII).** On InSDN and CICIDS2017, random flow splits give ≥ 0.9917 balanced accuracy and source-host-held-out splits give 0.907 ± 0.085 and 0.641 ± 0.056. We log the hosts of every split, decompose the InSDN dispersion by dominant host (leave-one-host-out), and test deduplication. This confirms, on an SDN benchmark, leakage mechanisms described for other datasets [14], [15]; it is not a new principle.
5) **Literature audit, recipes and forwarding mechanism (§IX, §X).** A targeted audit of 12 SDN-IDS papers, four feature recipes approximating published proposals, and a Mininet/Ryu case study of how forwarding granularity and idle timeout change controller-visible telemetry.

Scope and non-claims. We do not claim a deployable detector, evidence about controllers other than ONOS or datasets other than InSDN (CICIDS2017 is a non-SDN reference for the split analysis), or causal identification of the host effect. Section XII lists the threats to validity.

The remainder of this paper is organized as follows. Section II reviews background and related work. Section III formalizes the threat model, telemetry semantics, and problem formulation. Section IV examines controller observability in InSDN. Section V details the controller-observable detection pipeline, and Section VI presents detection results. Section VII analyzes benchmark session confounds. Section VIII presents the evaluation-validity audit on InSDN and CICIDS2017. Section IX details the Mininet/Ryu forwarding-application case study. Section X presents the targeted literature audit and feature-recipe evaluations. Section XI outlines methodological guidelines and an evaluation checklist. Section XII discusses limitations, and Section XIII concludes the paper.

II. Background and Related Work

A. SDN Control-Plane Mechanics and Telemetry

In an OpenFlow 1.3 network [11], switches maintain flow tables containing match rules, priorities, timeouts, and actions. When a packet arrives at an OpenFlow switch without matching an existing flow entry, it triggers a table-miss event. Under reactive forwarding configurations, the switch encapsulates the packet header (or the entire frame) into an asynchronous `OFPT_PACKET_IN` message and transmits it across the secure control channel to the controller. The controller's forwarding logic processes the packet headers, determines the forwarding path, and responds with an `OFPT_PACKET_OUT` message to forward the buffered frame. Concurrently, the controller may issue an `OFPT_FLOW_MOD` message to install a matching flow entry in the switch's flow table.

Subsequent packets belonging to the same flow match the installed entry directly in hardware or kernel space (such as Open vSwitch datapath), bypassing the controller entirely. The controller can monitor data-plane activity by issuing multipart `OFPMP_FLOW` requests to poll flow statistics. Switches respond with `OFPMP_FLOW` replies containing packet and byte counters for currently active flow entries. When an entry exceeds its idle or hard timeout, the switch evicts it and may send an `OFPT_FLOW_REMOVED` notification.

Crucially, the granularity of information visible to the controller via flow statistics is entirely dictated by the match fields specified in preceding FlowMods by the forwarding application. If an application installs coarse MAC-pair entries (`in_port, eth_src, eth_dst`), all transport connections and IP flows between those two endpoints aggregate into a single counter. Conversely, if an application installs granular 5-tuple entries, flow table occupancy increases dramatically, exposing switches to flow-table saturation attacks.

B. Pitfalls in Machine Learning Security and Network Datasets

The vulnerability of supervised machine learning to subtle experimental artifacts in computer security was seminalized by Sommer and Paxson [12]. More recently, Arp et al. [13] systematized ten common pitfalls in security ML, emphasizing sampling bias, data snooping, and spurious correlations.

In network intrusion detection specifically, benchmark datasets have faced substantial scrutiny. Engelen et al. [14] discovered critical flow construction, labeling, and TCP sequence errors in CICIDS2017, explicitly warning that non-statistical identifier columns (such as Flow IDs, IP addresses, and timestamps) allow classifiers to memorize host identities rather than attack patterns. Lanvin et al. [21] and Liu et al. [22] demonstrated that correcting label and orchestration bugs significantly shifts published model rankings. Flood et al. [15] surveyed seven benchmark datasets (excluding SDN datasets) and introduced the concept of "bad design smells," proving that standard randomized train/test splits result in widespread cross-split leakage due to synthetic traffic homogeneity. Roy et al. [16] demonstrated analogous shortcut learning dynamics on IoT intrusion benchmarks.

C. Intrusion Detection in SDN Environments

Early SDN security research focused on leveraging control-plane events to mitigate volumetric denial-of-service attacks. Braga et al. [23] proposed lightweight DDoS detection using 6-tuple flow statistics polled from NOX controllers. Mousavi and St-Hilaire [24] utilized PacketIn entropy variations for early DDoS detection. To protect the SDN control plane against saturation attacks, architectures such as AVANT-GUARD [25], FloodGuard [26], and SPHINX [27] introduced connection migration and flow-table constraint validation.

With the release of benchmark datasets such as InSDN [3], research shifted heavily toward applying advanced deep learning models to flow-level CSV features. Ataa et al. [4] evaluated CNN-LSTM and Transformer architectures on InSDN, reporting accuracies up to 99.02%. Saykat et al. [8] proposed OptiXGB-IDS, combining WGAN-GP oversampling with XGBoost to achieve 99.87% accuracy on InSDN. Mehmood et al. [10] reported 99.98% accuracy using optimizer-equipped CNN-MLPs. Ashikuzzaman et al. [5] introduced SDNGuard-Stack, reporting 99.98% accuracy using stacked ensemble learning. However, as we document in Section X, these works evaluate models exclusively on offline flow records and rely on randomized flow-level splits, leaving their operational validity unexamined.

III. Threat Model and Problem Formulation

A. System Architecture and Telemetry Semantics

Consider an SDN architecture comprising a set of OpenFlow switches $\mathcal{S}$ controlled by a logically centralized controller $\mathcal{C}$ over OpenFlow 1.3 control channels. The data plane carries traffic between end hosts $\mathcal{H} = \mathcal{H}_{\text{benign}} \cup \mathcal{H}_{\text{attack}}$.

The control channel between switch $s \in \mathcal{S}$ and controller $\mathcal{C}$ carries a discrete sequence of timestamped OpenFlow messages:

$$\mathcal{M} = \{(t_i, \tau_i, m_i)\}_{i=1}^{N}, \tag{1}$$

where $t_i \in \mathbb{R}^+$ denotes the arrival timestamp, τ_i denotes the message type belonging to the control-plane set $\{\text{PacketIn}, \text{PacketOut}, \text{FlowMod}, \text{FlowRemoved}, \text{StatsReply}\}$, and m_i

represents the message payload. When τ_i = PacketIn, m_i encapsulates the arriving frame header $h_i = (\text{mac}_{\text{src}}, \text{mac}_{\text{dst}}, \text{eth_type}, \text{ip}_{\text{src}}, \text{ip}_{\text{dst}}, p_{\text{src}}, p_{\text{dst}}, \text{proto})$. When τ_i = StatsReply, m_i conveys flow table records $e_{i,j} = (\text{match}_{i,j}, \text{pkts}_{i,j}, \text{bytes}_{i,j}, \text{dur}_{i,j})$.

B. Problem Formulation: Streaming Window Detection

Rather than assuming omniscient offline flow records, a controller-side detector operates on discrete time windows of duration W. For each window k spanning $[kW, (k+1)W)$, the detector computes a feature vector:

$$\mathbf{x}_k = \phi(\mathcal{M} \cap [kW, (k+1)W)) \in \mathbb{R}^d, \tag{2}$$

where $\phi(\cdot)$ extracts observable control-plane metrics. Given a history of H preceding windows, the model infers a binary or multi-class label $\hat{y}_k \in \{0, 1\}$.

C. Data Partitioning Formulations

To evaluate model generalization, we formalize four partitioning strategies over a dataset $\mathcal{D} = \{(\mathbf{x}_n, y_n, h_n)\}_{n=1}^{|\mathcal{D}|}$, where $h_n \in \mathcal{H}$ denotes the host associated with record n:

1) **Random Flow Split:** Partitions $\mathcal{D}$ into disjoint subsets $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$ via uniform random sampling. Crucially, the host sets overlap heavily: $\text{Hosts}(\mathcal{D}_{\text{train}}) \cap \text{Hosts}(\mathcal{D}_{\text{test}}) \approx \mathcal{H}$.
2) **Held-Out Host Split:** Partitions the host space $\mathcal{H}$ into disjoint subsets $\mathcal{H}_{\text{train}}$ and $\mathcal{H}_{\text{test}}$ such that $\mathcal{H}_{\text{train}} \cap \mathcal{H}_{\text{test}} = \emptyset$. All records belonging to $\mathcal{H}_{\text{train}}$ form $\mathcal{D}_{\text{train}}$, and records belonging to $\mathcal{H}_{\text{test}}$ form $\mathcal{D}_{\text{test}}$.
3) **Leave-One-Host-Out (LOHO):** For each host $h^* \in \mathcal{H}$, the test set is defined strictly as $\mathcal{D}_{\text{test}} = \{(\mathbf{x}_n, y_n, h_n) \mid h_n = h^*\}$, while the training set comprises all remaining hosts $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_n, y_n, h_n) \mid h_n \neq h^*\}$.
4) **Time-Ordered Split:** For a temporal dataset, records are ordered by timestamp $t_1 \leq t_2 \leq \cdots \leq t_N$, and a cutoff T_{split} defines $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_n, y_n) \mid t_n < T_{\text{split}}\}$ and $\mathcal{D}_{\text{test}} = \{(\mathbf{x}_n, y_n) \mid t_n \geq T_{\text{split}}\}$.

D. Threat Model and Scope

Defender Capabilities: The defender deploys an intrusion detection module co-located with or tapping the SDN controller interface. The detector inspects solely OpenFlow control-plane traffic $\mathcal{M}$. It possesses no packet taps on data-plane switch ports and cannot read host-side sockets. During any given evaluation epoch, the forwarding application running on the controller is assumed fixed and known, though operational changes or policy adjustments require recalibrating detector thresholds as discussed in Section IX.

Attacker Model: The attacker controls one or more compromised end hosts in the network. The attacker initiates network attacks including volumetric flooding (DoS/DDoS), port scanning, service probing, brute-force authentication, web application exploitation, and botnet command-and-control communication. The attacker does not compromise the controller kernel or directly forge control-channel TLS certificates.

Out of Scope: Real-time inline mitigation latency, hardware switch TCAM exhaustion, and adversarial evasion of control-plane telemetry features are outside the scope of this study.

IV. What the Controller Sees in InSDN

A. Dataset Inventory and Capture Reassembly

InSDN [3] was generated in a virtualized testbed using Mininet and Open vSwitch (OVS) connected to an ONOS controller [28]. Traffic targets a Metasploitable-2 virtual machine or the OVS switch itself. In addition to releasing CSV flow files, the authors published raw PCAP archives recorded at the target hosts and at the ONOS controller's OpenFlow interface.

We fetched all 44 ONOS controller packet captures directly from the institutional repository, verifying each archive against published SHA-256 digests. As summarized in Table III, the 44 captures encompass 11 benign sessions and 33 attack sessions spanning seven distinct attack classes.

B. Match Field Granularity and FlowMod Analysis

To determine what flow entries were active at the switches, we parsed and decoded every OpenFlow 1.3 multipart flow-statistics reply across all 44 controller captures. Over the entire dataset, only two distinct match structures appear:

1) `eth_type`: Default punting rules installed by ONOS (80 464 entry observations). Across all 44 captures, our full flow-table decode shows that these 80 464 entries partition equally across exactly four EtherTypes (20 116 observations each): IPv4 (`0x0800`), ARP (`0x0806`), LLDP (`0x88cc`), and BDDP (`0x8942`). All 80 464 default entries execute the explicit action `OFPAT_OUTPUT: OFPP_CONTROLLER` (controller port `0xfffffffd`).
2) `(in_port, eth_src, eth_dst)`: Reactive MAC-pair forwarding rules (24 437 entry observations). All 24 437 entries specify `OFPAT_OUTPUT` to physical switch data ports.

Not a single entry in any flow-statistics reply matches on source IP, destination IP, transport protocol, or transport port numbers. A 5-tuple flow—the fundamental unit labelled in InSDN's CSV—is never represented as an individual counter within the controller's flow table.

To verify whether transient 5-tuple entries could have been installed and deleted between polling intervals, we analyzed all 3 827 FlowMod messages transmitted by the ONOS controller. 3 811 FlowMods match strictly on `(in_port, eth_dst, eth_src)`, 16 match on `eth_type`, and exactly zero FlowMods specify IP addresses or transport ports. These FlowMods are consistent with an Ethernet learning switch; IPv4 reaches the controller through the punt rule described below.

C. Dynamics of the Controller PacketIn Stream

The controller's visibility of data traffic comes from PacketIns generated by ONOS's default punting rules and by reactive-forwarding misses. The flow-table decode shows an `eth_type=0x0800`→CONTROLLER rule in all 44 captures: all 20 116 observations have priority 5, below the MAC-pair rules (all 24 437 observations at priority 10), while the ARP, LLDP and BDDP punt rules have priority 40 000. Only 3 827 FlowMods occur in the 44 captures, and five captures contain none (Metasploitable DDoS SYN and TCP, and the three OVS

web-attack captures); their flow-statistics replies nevertheless show MAC-pair entries (4–73 observations per capture), which were installed before the capture started. We did not establish why packets that such entries should cover still reach the controller; candidates are traffic that matches no installed pair (spoofed or changing address combinations, one-way traffic) and ONOS applications that request packets, but we tested none of them. In Table IV and Fig. 2, we pair target-side packet counts with controller-side PacketIn volume across all attack scenarios.

In 9 of 12 attack scenarios, the controller receives between 0.68 and 1.29 PacketIns per data packet recorded at the target. Two Metasploitable-2 scenarios show 3.9 and 15.7. The OVS DDoS scenario shows 0.08 (6.5M target packets against 541k PacketIns), consistent with an untested hypothesis of control-channel saturation under a spoofed flood; we did not test this directly. The two sides are separate recordings and their alignment is unverified, so ratios are indicative, not exact per-packet counts. The Normal group has no target-side captures, so we have no such ratio for benign traffic. Because each PacketIn carries the packet's headers, the 5-tuple is visible to the controller per packet, but never as a per-flow counter in the switch flow table.

Software flow reconstruction from PacketIns. Because PacketIns carry packet headers, a monitor could try to reassemble 5-tuple flow records in software. We implemented a streaming reassembler (symmetric 5-tuple key, 120 s timeout, FIN/RST termination, i.e. CICFlowMeter's defaults) and ran it on one capture per scenario, comparing the result with the benchmark CSV (Table I, `packetin_reconstruction.json`). Three observations stand out. (i) In spoofed volumetric attacks and scans almost every reconstructed flow is a single packet (OVS DDoS 100.0%, OVS Probe 97.3%). The target-side traces show the same structure (6 510 923 packets over 6 508 402 distinct 5-tuples for OVS DDoS, Table IV), so this reflects the traffic and not information lost in the control channel. The benchmark CSV, by contrast, contains no single-packet flow in any of the six subsets examined, and for the two OVS captures we found no 5-tuple in common with it (Protocol and ports are zero in all OVS DDoS rows; for OVS Probe the cause is not established), so no accuracy statement is possible there. (ii) Where flows can be matched, rank agreement of packet counts is high on probing ($\rho = 0.86$) and benign traffic ($\rho = 0.94$) but weak on the DoS capture ($\rho = 0.24$, covering 1.7% of its reconstructed flows), and reconstructed packet totals are 38% (DoS) and 45% (benign) of the host-side totals, so the controller sees only a fraction of the packets of the flows it reassembles; on probing the totals agree (ratio 1.00). (iii) Durations are not reliably recovered: rank correlation is 0.98 on the benign capture, but -0.27 on DoS and -0.11 on probing, where almost all durations are near zero. We did not test why durations diverge under flooding (queueing or buffering of PacketIns is one candidate). The evidence shows only that, with this reassembler and these captures, software reconstruction recovers packet counts and durations in some scenarios and not in others. Matched flows are a subset of the reconstructed flows (Table I), the benign row is one of eleven captures, and the CSV is a separate recording whose alignment with the controller capture is unverified.

TABLE I
FLOWS REASSEMBLED FROM PACKETINS (120 S TIMEOUT, FIN/RST TERMINATION) AGAINST THE BENCHMARK CSV FOR ONE CAPTURE PER SCENARIO. CORRELATIONS AND THE PACKET RATIO USE ONLY MATCHED 5-TUPLES AND ARE OMITTED (–) BELOW 30 MATCHES. RATIO = RECONSTRUCTED OVER HOST-SIDE PACKET TOTAL.

Scenario	Flows	1-pkt %	Matched	Matched %	ρ_{pkt}	Ratio	ρ_{dur}
OVS DDoS	17,566	100.0	0	0.0	–	–	–
OVS Probe	11,293	97.3	0	0.0	–	–	–
OVS Web	61	37.7	8	19.7	–	–	–
MS2 DoS	29,818	7.1	494	1.7	0.24	0.38	-0.27
MS2 Probe	13,589	0.6	13,214	97.7	0.86	1.00	-0.11
Normal (1 of 11)	501	35.3	140	65.9	0.94	0.45	0.98

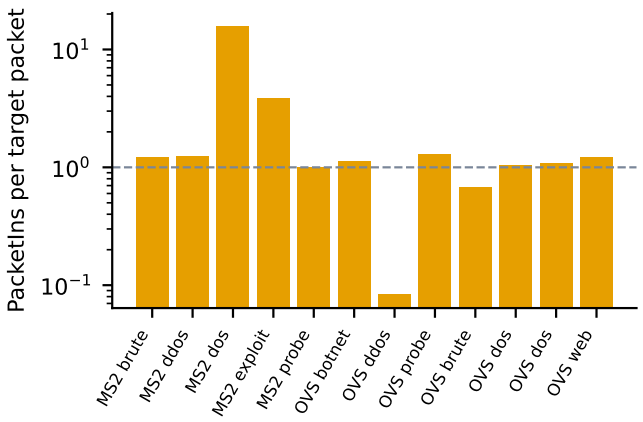


Fig. 2. PacketIns received by the ONOS controller per packet seen at the target host across InSDN attack scenarios (MS2: Metasploitable-2 target, OVS: OVS switch target). The dashed line marks parity (one PacketIn per target packet). Data: `gap_attack_v2.json`.

V. CONTROLLER-OBSERVABLE INTRUSION DETECTION

A. Streaming OpenFlow 1.3 Parser

We implemented a streaming OpenFlow 1.3 parser designed to reassemble TCP streams on port 6653. The parser handles message boundary segmentation and employs heuristic desynchronization recovery. If a message header is corrupted or desynchronized, the parser scans forward until it discovers two consecutive valid OpenFlow headers. Across all 44 captures, the parser processed 7 381 679 messages, requiring only 134 resynchronizations and discarding 12 217 bytes in total.

B. Feature Engineering

For consecutive non-overlapping windows of duration $W \in \{1\text{s}, 5\text{s}\}$, we extract 22 controller-observable features:

- **Control Message Counts (5):** Total counts of PacketIn, PacketOut, FlowMod, FlowRemoved, and FlowStats replies.
- **Header Diversity and Distributions (9):** Distinct source MACs, destination MACs, source IPs, destination ports; fractions of TCP, UDP, ICMP, and ARP traffic; and mean frame length.

- **Protocol and Entropy Metrics (3):** TCP SYN-only packet fraction, Shannon entropy of source IPs, and Shannon entropy of destination ports.
- **Flow Table Counter Deltas (5):** Number of active entries reported in statistics replies, sum and maximum of packet deltas, and sum and maximum of byte deltas between consecutive polls.

To capture temporal dynamics, the feature vector concatenates the current window with three preceding windows ($H = 4$), applying a $\log(1 + x)$ transformation to normalize dynamic ranges.

C. Strict Ground-Truth Window Labeling

Assigning ground-truth labels to controller windows requires precision. Marking every window within an attack capture as malicious would mislabel idle periods. Instead, we match the 5-tuple header of every PacketIn message against InSDN's ground-truth CSV flow records.

A window is labelled as an attack window if and only if it contains at least one PacketIn matching a known attack flow. In attack captures, windows containing unmatched IP traffic are marked as ambiguous and excluded from training and evaluation. Benign captures retain all windows. As shown in Table III, this strict protocol yields a sparse set of verified attack windows for low-volume classes (e.g., 6 windows for Brute Force, 12 for Exploit).

Two consequences of this construction shape everything that follows. First, a label requires the CSV to carry a usable 5-tuple. In `OVS.csv` all 48 413 DDoS rows have Protocol, source port and destination port set to 0, so no PacketIn can match them, and the three OVS DDoS captures contribute no attack windows. The two OVS brute-force captures also contribute none: their CSV rows target the container address 172.17.0.2 and we found no PacketIn with a matching 5-tuple (the cause is not established). Second, the labelled attack windows are dominated by probing: Table II shows that probe windows are 75.9% of all 2 589 attack windows, and the two Metasploitable-2 probe captures alone supply 1 922. Pooled scores are therefore close to probe-detection scores, and per-class results must be read with these counts in mind. By construction, attack windows are windows in which at least one CSV-matched flow crossed the controller; this selection may favour easy windows, and we cannot quantify that effect here.

D. Evaluated Models and Baselines

We evaluate several detector architectures:

- **Multi-Layer Perceptron (MLP):** A fully connected network (64–32 units, ReLU activation, dropout rate 0.2, class-weighted cross-entropy loss, Adam optimizer, 40 epochs).
- **Linear Classifier:** Logistic regression with L2 regularization operating on the same historical feature inputs.
- **Single-Window MLP:** An MLP evaluating only the current 1 s window without temporal history.
- **Feature-Ablated MLPs:** Models trained exclusively on PacketIn-derived features or flow-statistics features.

TABLE II
STRICTLY LABELLED CONTROLLER WINDOWS (W=1 S) PER DEPLOYMENT AND CLASS. A CLASS WITH ZERO ATTACK WINDOWS CONTRIBUTES ONLY ATTACK-FREE WINDOWS: OVS DDOS AND OVS BRUTE-FORCE HAVE NO CSV-MATCHED FLOWS (SECTION V). PROBE SUPPLIES 75.9% OF THE 2,589 POOLED ATTACK WINDOWS.

Deployment	Class	Captures	Windows	Kept	Attack win.
Metasp.	Brute force	2	45	33	6
Metasp.	DDoS	3	290	101	55
Metasp.	DoS	2	333	308	133
Metasp.	Exploit	4	88	71	12
Metasp.	Probe	2	1,989	1,956	1,922
OVS	Botnet	1	160	152	83
OVS	Brute force	2	559	326	0
OVS	DDoS	3	209	83	0
OVS	DoS	9	528	497	323
OVS	Probe	2	3,979	965	43
OVS	Web	3	636	275	12
Normal	Normal	11	33,283	33,283	0

- **Threshold Heuristics:** Univariate decision thresholds fitted on training folds (PacketIn rate, normalized burst rate, source-IP entropy, destination-port entropy).
- **Gradient Boosted Trees (XGBoost):** An optimized XGBoost baseline trained on the GPU using the same window features.
- **Host-Side Flow Baseline:** XGBoost trained on InSDN's 78 numeric CICFlowMeter features, which include the source and destination port numbers, under standard 70:30 flow splits. The audit of Section VIII repeats the split without ports, IP addresses, Flow ID and timestamps (77 features).

E. Validation Protocols

All evaluations enforce capture-level separation: no capture file appears in both training and testing sets. Metrics are reported across 10 random seeds as mean with 95% t-interval over seeds. For P1 each seed draws a new capture-to-fold assignment and a new network initialization, so this interval reflects both and not the sampling of captures; the capture-level cluster bootstrap in Table VI addresses the latter. The three detection protocols are:

- **P1 (Capture-grouped 5-fold CV):** Captures are partitioned into five folds, stratified by class.
- **P2 (Withheld attack families):** Entire attack families are held out (DoS and DDoS pooled as flooding); benign captures are split 70:30 between training and testing.
- **P3 (Cross-deployment transfer):** Training on Metasploitable-2 sessions and testing on OVS sessions, and vice versa; benign captures are split 70:30 as in P2.

We report Balanced Accuracy (BAcc), Recall, and False Positive Rate (FPR). The evaluation-validity audit of Section VIII uses its own protocols A–E.

VI. EMPIRICAL DETECTION RESULTS

A. Performance on Unseen Captures

Table V presents detection results on unseen captures under 5-fold cross-validation (W=1 s). Across 10 seeds (Section V),

the controller-observable MLP achieves a balanced accuracy of 0.946 ± 0.016 with a false positive rate of 0.001. Logistic regression performs competitively at 0.939, and the single-window MLP reaches 0.934, indicating that temporal history provides marginal separation gain in this feature space.

In contrast, standard SDN heuristics perform poorly: the PacketIn-rate threshold achieves only 0.616 balanced accuracy (FPR 0.561), failing because benign interactive sessions also generate bursty PacketIn streams. Entropy thresholds perform substantially better (0.839–0.852). Ablation confirms that PacketIn features drive performance (0.943 alone), while flow-statistics features alone achieve only 0.699 with an FPR of 0.284, consistent with coarse MAC-pair aggregation.

While Table V reports variation across seeds (± 0.016; each seed redraws the capture-to-fold assignment and the initialization), a cluster bootstrap resampling over captures (Table VI) measures capture-level variance, yielding a pooled BAcc of 0.984 [0.926, 0.995] for the MLP. A label-shuffle sanity check produces a balanced accuracy of 0.512 ± 0.054 at 1 s and 0.508 ± 0.067 at 5 s, i.e., chance level; this confirms that the featurization pipeline does not inadvertently encode label information directly, though it does not address session-level confounding.

B. Withheld Attack Families and Cross-Deployment Transfer

Under leave-one-attack-family-out testing (Table VII), the controller-side MLP reaches 0.936 on floods, 0.997 on probes and 0.904 on web attacks, above the best single-feature threshold in those families; these scores are subject to the session confound of Section VII. On sparse minority classes, thresholds or linear models match or exceed it (e.g., botnet threshold 0.965 vs. MLP 0.932; logistic regression 0.997 on exploit). The brute-force, exploit and web families rest on only 6, 12 and 12 attack windows (Table II), so their scores carry uncertainty that the seed interval does not capture. In cross-deployment evaluations (Table VIII), training on OVS sessions and testing on Metasploitable-2 yields 0.990 balanced accuracy, while the reverse direction achieves 0.905 (with logistic regression reaching 0.928). The OVS→Metasploitable-2 direction is scored on 2 128 attack windows, 1 922 (90%) of them probe windows; the reverse direction is scored on 461 OVS attack windows (botnet 83, DoS 323, probe 43, web 12). Extending the window length to $W=5$ s yields consistent performance (MLP 0.958, logistic regression 0.961, XGBoost 0.952).

C. Comparison with Host-Side Flow Baseline

For contrast, applying XGBoost to InSDN's host-side CICFlowMeter features (343 889 rows, 78 numeric features including the two port numbers) under standard randomized 70:30 flow splits yields a balanced accuracy of 0.9998 ± 0.00003. This near-perfect accuracy should not be read as evidence of detection ability: Section VIII shows that it does not survive host-held-out evaluation on this dataset.

TABLE III
CONTROLLER CAPTURES (1 S WINDOWS). UNDER STRICT LABELS, AMBIGUOUS WINDOWS OF ATTACK CAPTURES ARE EXCLUDED ("KEPT").

Class	Captures	Windows	Kept	Attack win.	PacketIns
Normal	11	33,283	33,283	0	789,569
DoS	11	861	805	456	845,362
DDoS	6	499	184	55	1,340,347
Probe	4	5,968	2,921	1,965	621,986
Brute force	4	604	359	6	5,564
Exploit	4	88	71	12	332
Web	3	636	275	12	1,199
Botnet	1	160	152	83	1,002

TABLE IV
TARGET-SIDE TRAFFIC VS. CONTROLLER OBSERVATIONS PER SCENARIO (RATIO = PACKETINS PER TARGET PACKET).

Group	Scenario	Target pkts	5-tuples	PacketIns	Ratio
Metasp.	Brute force	1,311	295	1,574	1.20
Metasp.	DDoS	652,495	651,083	799,253	1.22
Metasp.	DoS	13,078	989	205,198	15.69
Metasp.	Exploit	86	18	332	3.86
Metasp.	Probe	507,650	51,150	510,194	1.01
OVS	Botnet	902	164	1,002	1.11
OVS	DDoS	6,510,923	6,508,402	541,094	0.08
OVS	Probe	86,705	79,021	111,792	1.29
OVS	Brute force	5,892	1,112	3,990	0.68
OVS	App. DoS	299,625	41,738	308,681	1.03
OVS	Net. DoS	304,886	20,979	331,483	1.09
OVS	Web	987	176	1,199	1.21

VII. VALIDITY ANALYSIS: DISENTANGLING ATTACKS FROM SESSION ARTIFACTS

A. The Trivial IP PacketIn Rule

In the distributed captures, benign and attack traffic were recorded in separate sessions, and the attack sessions contain no identifiable concurrent benign background traffic (see the window counts below). Strict labelling therefore confounds session type with the label.

When we evaluate the MLP strictly within attack sessions, it achieves a pooled balanced accuracy of 0.942 (macro-averaged 0.906). However, a trivial baseline rule—"classify window as attack if it contains at least one IP PacketIn"—achieves a pooled balanced accuracy of 0.995 and a macro-averaged score of 1.000 across the exact same windows (Fig. 3). This 0.995 score reflects the labeling protocol combined with session separation: because the ambiguity mask excludes attack-session windows containing unlabelled IP traffic, the surviving attack-free windows contain almost no IP traffic. Of the 2 178 attack-free windows kept in attack sessions, 1 858 are empty (no PacketIn at all) and are excluded from this metric; of the remaining 320, 317 contain no IP PacketIn (ARP, LLDP or other non-IP frames). The 0.995 therefore rests on 2 589 attack windows and 320 attack-free windows. Thus, any rule checking for the presence of IP packets achieves near-perfect separation inside attack sessions without learning any attack characteristics.

Operational Evaluation across Benign Traffic. While the trivial rule exploits this non-interleaved benchmark artifact within attack sessions, evaluating

TABLE V
UNSEEN CAPTURES, GROUPED 5-FOLD CV, W=1 S, STRICT LABELS, 10 SEEDS (MEAN±95% CI OVER SEEDS).

Method	BAcc	Recall	FPR
MLP, all features	0.946±0.016	0.893	0.001
MLP, PacketIn features only	0.943±0.015	0.887	0.002
Logistic regression (history)	0.939±0.016	0.887	0.009
MLP, single window	0.934±0.015	0.870	0.003
Thr. source-IP entropy	0.852±0.029	0.763	0.059
Thr. dst-port entropy	0.839±0.018	0.736	0.059
MLP, flow-stats features only	0.699±0.036	0.683	0.284
Thr. PacketIn burst	0.643±0.008	0.912	0.627
Thr. PacketIn rate	0.616±0.011	0.793	0.561
MLP, within attack sessions	0.942±0.016	0.893	0.008
macro-averaged over captures	0.906±0.009	–	–
Trivial: "any IP PacketIn" (same windows)	0.995	–	–
macro over captures	1.000	–	–

TABLE VI
POOLED WINDOW-COUNT BACC WITH A CAPTURE-LEVEL CLUSTER BOOTSTRAP (2000 RESAMPLES, STRATIFIED BY CLASS).

Method	Pooled BAcc	95% CI
MLP, all features	0.984	[0.926, 0.995]
Logistic regression	0.979	[0.921, 0.991]
MLP, single window	0.979	[0.905, 0.993]
Thr. source-IP entropy	0.923	[0.783, 0.959]
Thr. dst-port entropy	0.926	[0.792, 0.953]
Thr. PacketIn rate	0.467	[0.245, 0.817]

it against operational benign traffic (the 11 Normal sessions, 33,283 windows) exposes its failure mode (`operational_stream_evaluation.json`): because 77.5% of normal-session windows contain IP PacketIns (25,810 of 33,283 windows), the trivial rule flags 77.5% of normal-session traffic as attacks (normal-session FPR = 0.775). Evaluated on the full strict labeled window set (38,050 windows: 2,589 attack, 35,461 pooled benign), every attack window contains an IP PacketIn by construction of strict labels (Recall = 1.000), but its overall benign FPR of 72.8% drops balanced accuracy to 0.6360 (and to 0.4889 across the raw unmasked 42,099-window stream). In contrast, the controller-observable MLP avoids triggering on benign IP traffic, achieving a false positive rate of only 0.001 on normal sessions and 0.946 balanced accuracy overall (Table V). However, as our session probes demonstrate below, avoiding simple heuristic triggers does not mean the MLP has learned invariant attack semantics: classifiers trained on residual traffic stripped of all attack PacketIns still predict session origin with 0.940 balanced accuracy (Table IX). Thus, non-interleaved benchmark recordings make it difficult to untangle genuine attack detection from controller-level session fingerprints.

B. Host-Identity Baseline

To bound how much of the controller-side score can be explained by *which hosts appear*, without any traffic statistics, we evaluate two rules learned on the training captures of each fold (same folds, labels, windows and seeds as Table V). Rule R1 predicts attack if the window contains a PacketIn source address that appeared in training attack windows and in no training attack-free window; rule R2 predicts attack if it contains any source address seen in training attack windows (`session_identity_baseline.json`). Table XIII shows that R1 reaches a balanced accuracy of 0.734 over all strictly labelled windows, with recall limited to 0.472 because the source addresses of spoofed floods never repeat across captures, and that R2 (0.680) is dominated by false positives on benign sessions. Within attack sessions R2 is again near-perfect (0.994), like the any-IP rule.

TABLE VII
WITHHELD ATTACK FAMILY (BACC, W=1 S, STRICT LABELS). "BEST THR.": BEST OF FOUR THRESHOLDS, CHOSEN PER FAMILY.

Withheld family	MLP	Logistic	Best thr.
Flood (DoS+DDoS)	0.936±0.011	0.953	0.814
Probe	0.997±0.000	0.988	0.970
Web	0.904±0.020	0.909	0.699
Brute force	0.891±0.029	0.913	0.888
Exploit	0.929±0.037	0.997	0.929
Botnet	0.932±0.024	0.803	0.965

TABLE VIII
CROSS-DEPLOYMENT TRANSFER (BACC, STRICT LABELS).

Train → test	MLP	Logistic	Best thr.
Metasploitable-2 → OVS	0.905±0.021	0.928	0.866
OVS → Metasploitable-2	0.990±0.001	0.985	0.947

Host identity therefore explains a substantial part of the separation but not all of it: the MLP exceeds R1 by 0.21 balanced accuracy, so its score is not reducible to address memory. However, the session probe below still separates sessions at 0.940 after attack-matched PacketIns are removed, so we cannot attribute the remaining margin to attack semantics rather than to other session fingerprints such as traffic mix or timing. The rules are simple; a stronger identity model could score higher, so these numbers are lower bounds on what identity can explain.

C. Session Probes

To test whether classifiers recognize session identity rather than attack signatures, we trained an MLP on windows where all attack-matched PacketIns were excised (Table IX). The model predicts whether a window originates from an attack session with 0.940 balanced accuracy (D1). When restricted to attack-free windows (D2), it achieves 0.950. However, 82% of these residual windows contain unmatched IP traffic that likely represents unlabelled attack remnants, and D2 rests on only 6 windows from 2 captures. These findings demonstrate that in non-interleaved benchmark recordings, attack detection cannot be cleanly disentangled from session fingerprinting.

VIII. EVALUATION-VALIDITY AUDIT: RANDOM FLOW SPLITS VS. HELD-OUT HOSTS

A. Experimental Design Across Two Benchmarks

To evaluate leakage in flow-based evaluations, we implemented a standardized evaluation pipeline applied to two prominent benchmarks: InSDN (343 889 flows) and an external

TABLE IX
SESSION PROBE: PREDICTING BENIGN VS. ATTACK *session* FROM PACKETIN FEATURES AFTER REMOVING ATTACK-MATCHED PACKETINS (≥5 REMAINING PER WINDOW). THE RESIDUAL CAN STILL CONTAIN UNLISTED ATTACK TRAFFIC (D1: 424 OF 518 ATTACK-SESSION WINDOWS CONTAIN UNMATCHED-IP PACKETINS), AND D2 RESTS ON 6 WINDOWS FROM 2 CAPTURES.

Variant	BAcc	Majority
All windows (D1)	0.940±0.005	0.500
Attack-free windows only (D2)	0.950±0.031	0.500

TABLE X
EVALUATION-VALIDITY AUDIT: BALANCED ACCURACY (BENIGN VS. ATTACK) OF THE SAME XGBOOST (GPU) ON THE 77 NUMERIC CICFLOWMETER FEATURES, SAME PIPELINE ON BOTH DATASETS, 10 SEEDS (MEAN±95% CI OVER SEEDS). A, B AND C USE NO IPS, PORTS OR TIMESTAMPS; D USES ONLY THOSE. ROWS: INSDN 343 889, CICIDS2017 2 100 814. C IS A SINGLE DETERMINISTIC SPLIT.

Protocol	InSDN	CICIDS2017
A. Random 70:30 flow split	0.9998	0.9917±0.0001
B. Held-out source hosts (70:30 by source IP)	0.907±0.085	0.641±0.056
C. Per-file time-ordered 70:30 (deterministic)	0.9997	0.7187
D. IPs, ports and Flow ID only, random split	1.0000	0.9999
E. Split-A test rows duplicating a train row (%)	48.7	20.3

reference dataset, CICIDS2017 (2 100 814 flows). We evaluate an identical GPU-accelerated XGBoost classifier across four distinct protocols using 10 random seeds (Table X and Fig. 4):

- **Protocol A (Random 70:30 Flow Split):** Standard randomized flow-level partition. Excludes all IP addresses, ports, Flow IDs, and timestamps (retaining 77 numeric features).
- **Protocol B (Held-Out Source Hosts):** 70:30 partition grouped by source IP. No source host appears in both training and test sets.
- **Protocol C (Time-Ordered Split):** Deterministic 70:30 chronological partition per capture file.
- **Protocol D (Identifier-Only Split):** Random 70:30 split trained exclusively on IP addresses, port numbers, and Flow IDs.
- **Protocol E (Redundancy Audit):** Measurement of the percentage of test-set rows that possess an exact feature duplicate in the training set under Protocol A.

B. The Generalization Collapse on Unseen Hosts

Under Protocol A (random flow split), XGBoost achieves near-perfect balanced accuracy: 0.9998 on InSDN and 0.9917 ± 0.0001 on CICIDS2017. However, under Protocol B (held-out source hosts), performance on InSDN drops by nine percentage points to 0.907 ± 0.085, with individual splits swinging from 1.00 down to 0.66. Across the 10 random seeds on InSDN, the balanced accuracy distribution is widely dispersed: five seeds score near 1.00 (0.996, 0.998, 0.999, 1.000, 1.000), two score between 0.85 and 0.94 (0.859, 0.939), and three drop below 0.84 (0.660, 0.786, 0.837), consistent with the acute host concentration documented below. On CICIDS2017, the drop is catastrophic: performance collapses by 35 percentage points to 0.641 ± 0.056. A per-file time-ordered split shows no drop on InSDN (0.9997) but falls to 0.719 on CICIDS2017. We did not analyze why; CICIDS2017's attacks were run in scheduled time slots [6], so a chronological cut can withhold an attack phase, which would be consistent with the drop.

Under Protocol D, a classifier supplied exclusively with network identifiers (IP addresses, ports, and Flow ID) achieves 1.0000 on InSDN and 0.9999 on CICIDS2017. Network identifiers alone therefore separate the classes in these benchmarks; this does not show that models trained on flow features use them, only that they could.

TABLE XI
FEATURE-RECIPE REPLICATION ON INSDN (ONE XGBOOST CLASSIFIER, GPU, 80:20, 10 SEEDS, MEAN±95% CI): BINARY BALANCED ACCURACY AND 8-CLASS ACCURACY UNDER A RANDOM FLOW SPLIT AND A SOURCE-IP-GROUPED SPLIT. RECIPES APPROXIMATE FEATURE CHOICES DESCRIBED IN PUBLISHED INSDN PAPERS; THEY ARE NOT THOSE PAPERS' MODELS OR CODE. GROUPING BY SOURCE HOST ALSO CHANGES THE ATTACK-CLASS MIX BETWEEN TRAIN AND TEST, SO GROUPED VALUES ARE A STRESS TEST.

Recipe	BAcc random	BAcc grouped	8-cls random	8-cls grouped
Numeric features only	0.9998±0.0000	0.943±0.057	0.9991±0.0001	0.788±0.186
Numeric + ports	0.9998±0.0000	0.856±0.151	0.9998±0.0000	0.687±0.241
Numeric + identifier columns	1.0000±0.0000	0.941±0.088	0.9999±0.0000	0.804±0.187
Top-15 by MI, selected before split	1.0000±0.0000	0.941±0.088	0.9999±0.0000	0.835±0.175

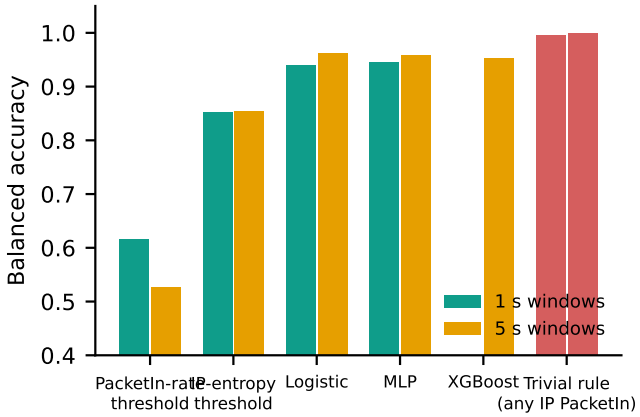


Fig. 3. Balanced accuracy on unseen captures under capture-grouped CV across 10 seeds (XGBoost evaluated at 5 s). The red bars represent the trivial rule ("at least one IP PacketIn"), which matches or outperforms all machine learning models within attack sessions.

C. The Role of Data Redundancy and Class Coverage

Under Protocol E, we discover that 48.7% of random-split test rows in InSDN (and 20.3% in CICIDS2017) are exact duplicates of training rows. To test whether duplicate flows drive the random-split score, we deduplicated InSDN completely, reducing the dataset from 343 889 to 184,607 rows. On this deduplicated dataset, the random-split score remains unchanged at 0.9998. Thus, feature redundancy alone does not explain the performance gap.

To ensure that host grouping does not artificially penalize performance by withholding entire attack classes from training, we enforced a class-coverage constraint: holding out source hosts while guaranteeing that every attack class appears in the training partition. Under this controlled setting, balanced

TABLE XII

TARGETED LITERATURE AUDIT OF REPRESENTATIVE SDN INTRUSION DETECTION EVALUATIONS ON BENCHMARK DATASETS (12 PAPERS EXAMINED, 10 READ IN FULL TEXT; DETAILS AND QUOTATIONS IN `LITERATURE_CHECK.JSON`). NONE OF THE INSDN PAPERS READ IN FULL REPORTS A HOST-GROUPED OR CAPTURE-GROUPED SPLIT, AND THREE USE IDENTIFIER ATTRIBUTES AS INPUT FEATURES.

Paper	Venue	Dataset	Evaluated Model	Split Protocol	Identifiers Used?	Methodological Findings / Red Flags
Towhid & Shahriar [29]	NOMS 2023	InSDN	RF / LSTM	Random 70:30	Excluded	Divides all flows 70:30; no host/session grouping; offline Scapy features
Ataa et al. [4]	Sci. Rep. 2024	InSDN	CNN-LSTM / Transf.	Shuffled (ratio unstated)	Excluded	Shuffled flows; no train/test ratio or grouping stated; single run
Abdallah et al. [30]	ACM ARES 2021	InSDN	Hybrid CNN-LSTM	Unverified (paywall)	Unverified	Publisher access restricted; split details unverified
Elsayed et al. [31]	ASEJ 2023	InSDN / ToN-IoT	SATIDS (LSTM)	Unverified (paywall)	Unverified	Publisher access restricted; split details unverified
Saykat et al. [8]	Sci. Rep. 2026	InSDN / CIC-17	OptiXGB + WGAN-GP	80:20 sample-level	Src/Dst Port kept	Preprocessing leakage controlled, but the split has no host grouping; Dst Port is the major split of the surrogate tree
Mujahid et al. [9]	PeerJ CS 2025	InSDN / Kaggle	RF + Deep Extractor	train_test_split (ratio unstated)	Unstated	scikit-learn row-level split; no grouping; row count mismatch unexplained
Mehmood et al. [10]	PLOS ONE 2025	InSDN / CICDDoS	CNN-MLP	Unstated	Dst Port kept	No split protocol stated; Dst Port is the strongest negative SHAP contributor
Alqhatani et al. [32]	PLOS ONE 2025	UNSW / BoT-IoT	GRU-NTM	Chronological 80:20	Unstated	Chronological split, but the 20% is a validation set used for early stopping/tuning
Ashikuzzaman et al. [5]	arXiv 2026	InSDN	Stacking + LightGBM	80:20 (method unstated)	Flow ID, IPs, Ports	Top-15 mutual-information features include Flow ID, Src/Dst IP, Src/Dst Port; SHAP ranks Flow ID first
Malik et al. [33]	Unreviewed ms. 2026	InSDN	BiLSTM + RFE	Stratified 5-fold CV	Unstated	Stratified 5-fold drops accuracy to 81.60%; attributed to split variance
Rohith et al. [34]	Sci. Rep. 2026	InSDN	Quantum + MSDC Net	80:20 (method unstated)	Unstated	Four split ratios tried; 80:20 chosen from the reported test results; no grouping
Saleh & Hamad [35]	JESA 2026	SICA / InSDN	DNN / LSTM	70:20:10 (method unstated)	Excluded	Removed Flow ID/IPs, yet InSDN scores 99.61% (split presumably random)

TABLE XIII

HOST-IDENTITY RULES AGAINST THE CONTROLLER-SIDE MLP, STRICT LABELS, W=1 S, CAPTURE-GROUPED 5-FOLD CV, 10 SEEDS (MEAN±95% CI OVER SEEDS). RULES ARE LEARNED ON TRAINING CAPTURES ONLY AND USE NO TRAFFIC STATISTICS.

Method	BAcc	Recall	FPR
All strictly labelled windows			
MLP, all features (Table V)	0.946±0.016	0.893	0.001
R1: attacker-only IP memory	0.734±0.033	0.472	0.004
R2: any attack IP	0.680±0.009	0.997	0.637
Within attack sessions (non-empty windows)			
MLP, all features	0.942±0.016	0.893	0.008
R1: attacker-only IP memory	0.736±0.032	0.472	0.000
R2: any attack IP	0.994±0.003	0.997	0.008

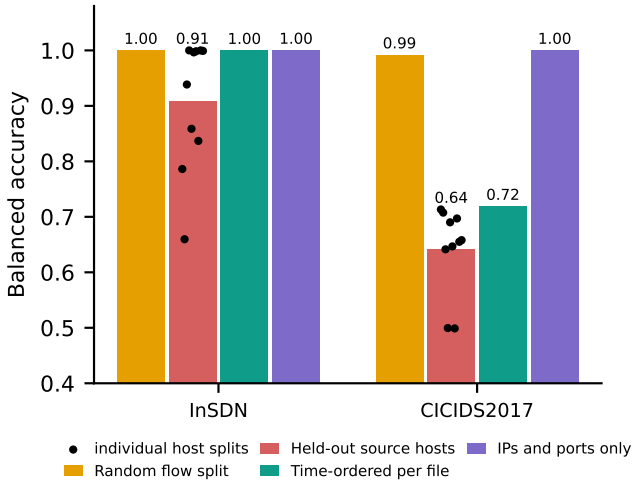


Fig. 4. Balanced accuracy across four evaluation protocols on InSDN and CICIDS2017 (10 seeds; dots represent individual host splits). When source hosts are held out, performance degrades sharply, and identifier-only models achieve near-perfect scores.

accuracy is 0.939 (0.812 after deduplication). Grouping by the 3-tuple (`src_ip, dst_ip, dst_port`) yields 0.961 (0.984 after deduplication). The triple-grouped split fragments traffic while retaining communication from the same attacker hosts on different ports across partitions, allowing the model to leverage host-level similarities without exact flow duplication.

D. Host Concentration and the Held-Out-Host Dispersion

Why do held-out-host scores vary so widely? InSDN contains 122,868 distinct source IP addresses; however, 122,307 of these source IPs appear in exactly one flow record, generated during spoofed DoS/DDoS flooding. Crucially, only eight hosts account for 500 or more flows, yet these eight hosts encompass 215,404 of InSDN's 343,889 total rows. Among these eight dominant hosts, exactly three source attack-labelled flows.

By logging host assignments across all 10 evaluated Protocol B seeds (`host_assignment_audit.json`), we observe that the balanced accuracy dispersion correlates with dominant host allocations:

- Assigning the primary attacker host `200.175.2.130` (110 799 flows, 32.2% of the dataset) to test (Seeds 0 and 1) elevates test share to 43.7% and 57.1%, dropping balanced accuracy to 0.786 and 0.660.
- When the primary benign host `192.168.20.133` (53 790 flows, 15.6% of the dataset) is assigned to test (Seeds 4 and 7), balanced accuracy drops to 0.859 and 0.837 due to false positives on unseen benign activity.
- When neither primary host is withheld, performance is generally high: Seeds 3, 5, 6, 8, and 9 score between 0.996 and 1.000 (test row share 11.2%–22.1%). The one intermediate case is Seed 2 (BAcc 0.939, test share 22.2%), where neither primary host is in test. Host `192.168.20.1`, which yields an 18.8% FPR when held out alone (leave-one-host-out, below), is in the test set of Seed 2 (and of Seed 0), which is consistent with the intermediate score; we have not verified the role of this host.

The 0.66–1.00 range is consistent with the uneven allocation of these few dominant hosts across splits, as are the leave-one-host-out results below. With ten correlated splits this is an association, not an identification of cause.

To quantify generalization to individual hosts, we conducted a Leave-One-Host-Out (LOHO) evaluation, holding out one dominant host at a time while training on all remaining records. When holding out the primary attacker (host 200.175.2.130), the model catches only 51.6% of its attack flows (DoS detection recall collapses to 34.7%). Conversely, when holding out the largest benign host (host 192.168.20.133), the false positive rate spikes to 37.9%. On CICIDS2017, which contains 48 source IPs (15 with $\geq$ 500 flows), holding out the primary attacker (172.16.0.1, 442 098 flows) results in an attack recall of only 28.3% and a benign false positive rate of 54.0%. Individual bot hosts held out in isolation achieve detection rates between 23.4% and 43.5%.

E. Replication of Published Pipelines

We partially replicated the published SDNGuardStack pipeline [5] using our own implementation across 5 seeds. Under a random split, the model reproduces its published performance, achieving 0.9999 accuracy (published: 99.98%). However, under held-out source hosts, mean accuracy drops to 0.72 when selecting features strictly on training folds (per-seed: 0.24, 0.73, 1.00, 0.66, 0.99) and 0.78 when selecting on all rows (per-seed: 0.33, 0.99, 1.00, 0.62, 0.98); 95% confidence intervals span up to ± 0.39. The metric is 8-class accuracy. This dispersion is consistent with InSDN's host concentration: a 20% host partition draws hosts with very unequal traffic volumes, the test row share varies from 8% to 40% across seeds, and the lowest score (0.24) occurs in the run with the largest test partition (40% of all rows), consistent with the test fold being dominated by a few large hosts while no test class is absent from training (`unseen_test_share` is 0.0 in all seeds). We did not test this attribution further.

As a second published pipeline we evaluated a *proxy* for Ataa et al. [4]. Their exact 48-feature set and architectures (a Transformer encoder and a CNN-LSTM) could not be reproduced from the text, so the proxy is a binary MLP on all numeric flow features; it is not their model, and their reported 99.02% is an 8-class accuracy that is not comparable with ours. Over 5 seeds the proxy reaches 0.9994 ± 0.0001 accuracy under a random split and 0.820 ± 0.202 under held-out source hosts (per-seed: 0.68, 0.71, 1.00, 0.71, 0.99; macro-F1 0.706 ± 0.299). The interval is again wide, consistent with the host concentration above. We make no claim about the original model.

IX. A Forwarding-Application Case Study

InSDN shows one forwarding configuration. To see whether what the controller observes depends on the forwarding application, we built a small testbed (Mininet, one Open vSwitch switch, six hosts; a Ryu OpenFlow 1.3 learning switch with a configurable entry granularity) and recorded its control channel with the same parser. The three modes install, for each learned destination, an entry matching (i) the MAC pair; (ii) the 5-tuple for TCP/UDP with a 10 s idle timeout; or (iii) the 5-tuple with a 2 s idle timeout applied to all entries. In the two 5-tuple modes, other IP traffic receives an (src, dst, protocol) entry and ARP an ARP-only entry, so a coarse entry for one protocol cannot match another. The controller polls flow statistics every second. Traffic is benign only and generated by the same program with the same random seeds in every mode (request timing still differs): five hosts repeatedly fetch files of 16 KB to 1 MB from an HTTP server (80%) or ping it (20%) with randomized uniform pacing (0.05–0.8 s) for 90 s (captures span ∼99 s including start-up). Each mode was run 10 times (seeds 0–9), reporting mean and 95% t-interval over runs (Table XIV); the 30 runs completed 26 891 requests, of which 41 failed (below). We repeated the study on a loop-free three-switch chain (Table XV, below).

Granularity sets the PacketIn volume. MAC-pair mode produced 12.3 ± 2.3 TCP PacketIns per run, against $10\,422 \pm 202$ (10 s idle) and $10\,392 \pm 249$ (2 s idle) under per-flow forwarding:

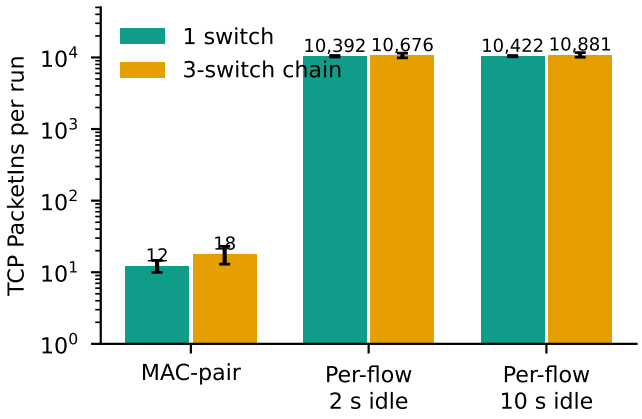


Fig. 5. TCP PacketIns per run (mean ± 95% CI, 10 runs per mode, log scale) under three forwarding modes, on one switch and on a three-switch chain. Data: `live_modes_v4_one10.json`, `live_modes_v4_linear3.json`.

665× to 1 752× (10 s idle) and 668× to 1 780× (2 s idle) more in each of the ten paired runs (larger in 10 of 10; two-sided sign-test $p = 0.002$). That is about 11–12 TCP PacketIns per completed request, so each connection is punted repeatedly; the most likely cause is packets that arrive before the FlowMod takes effect (we install entries without a barrier and send full packets), but we did not isolate this mechanism. FlowMods (19.1 vs. about 10 440 and 10 560 per run) track the PacketIns and are not independent evidence. The MAC-pair result is a replication: the same mode in the first, flawed campaign gave 11.0 ± 4.1. Seeds 0–4 are the earlier five runs per mode (reported in the previous version of this study); seeds 5–9 were added later.

Idle timeout sets what a poller sees. Per-flow entries returned per statistics reply fell from 153.1 ± 2.4 to 36.9 ± 0.5 when the idle timeout dropped from 10 s to 2 s, a ratio of 0.234 to 0.246 in all ten pairs: entries expire between polls, so a 1 s poller sees about a quarter as many. The result is close to what a shorter timeout implies; its value is the size of the loss for a realistic polling interval. In MAC-pair mode the replies contain only the table-miss rule and MAC-pair entries (9.5 ± 0.03 per reply); no entry matches IP or transport fields, as in InSDN (whose ONOS additionally installed `eth_type` punt rules).

A flawed first campaign. Our first application installed MAC-pair entries for ping and ARP in every mode; these also match later TCP between the same hosts, so most connections never reached the controller and per-flow mode produced only 43.4 ± 30.1 TCP PacketIns per run (4× MAC-pair) with a wide interval. The original application is archived as `ryu_fwd_v1.py`. We report it because it shows how an overlapping coarse entry hides traffic from the controller by two orders of magnitude, independently of the forwarding mode's nominal granularity.

Reliability. One run (5-tuple, 2 s idle, seed 4) had 41 failed requests (7–10 per client); the other 29 one-switch runs and all 30 three-switch runs had none. The run did not reproduce, and its failures were spread evenly over all five clients, which is consistent with a short stall of the shared path or controller rather than one client. The cause is not established (the failure

TABLE XIV
FORWARDING APPLICATION VS. WHAT THE CONTROLLER OBSERVES: BENIGN TRAFFIC FROM THE SAME GENERATOR AND SEEDS (5 HOSTS → 1 HTTP SERVER, PLUS PINGS), 1-SWITCH MININET/RYU TESTBED, ∼99 S CAPTURE PER RUN, 10 RUNS PER MODE, MEAN±95% CI. ENTRIES = FLOW ENTRIES RETURNED PER FLOW-STATISTICS REPLY (1 S POLLING).

Mode	TCP PacketIns	All PacketIns	FlowMods	MAC-pair entries	IP-specific entries
MAC-pair (as ONOS in InSDN)	12.3±2.3	76.8±1.8	19.1±1.8	9.5±0.0	0.0±0.0
5-tuple, 10 s idle	10422.0±202.0	10497.0±204.7	10442.8±200.8	0.0±0.0	153.1±2.4
5-tuple, 2 s idle	10391.6±249.4	10611.3±258.7	10556.4±252.3	0.0±0.0	36.9±0.5

reasons were not logged in that run; later runs log them). The run is included in all figures.

A second topology. On a three-switch chain (hosts h1–h6, server two hops from h1 and h2; 10 runs per mode, 26 826 requests, none failed) the contrast holds (Table XV): 18.0 ± 5.0 TCP PacketIns per run under MAC-pair against $10\,881 \pm 794$ and $10\,676 \pm 804$ under per-flow forwarding ($391\times$ to $1\,509\times$ in every paired run), and the per-flow entries a 1 s poller sees per reply fall from 112.4 ± 2.3 to 27.0 ± 0.4 (ratio 0.235 to 0.245). The ratio is smaller than on one switch because the MAC-pair baseline rises with the extra hops (18.0 vs. 12.3).

A stock Ryu application. To check that the MAC-pair behaviour is not an artefact of our own application, we ran Ryu's unmodified `simple_switch_13` on the same one-switch topology (5 runs of 60 s; 2 982 requests, none failed). Of 91 FlowMods, 86 match `(in_port, eth_dst, eth_src)` and 5 are the table-miss rule; none matches IP or transport fields. The controller received 71, 69, 76, 76 and 76 PacketIns per run (mean 73.6, all protocols). Like ONOS's `fwd`, this application installs MAC-pair entries; this follows from the application's code, and it does not test OpenDaylight, ONOS under other applications, or any production deployment.

Scope. This is a testbed with at most three switches with one forwarding application and benign traffic only; we run no attacks or detectors in it. The mode contrast does not transfer to InSDN's ONOS, which punted about one packet per target packet even with MAC-pair entries (§IV), and no claim is made about detection accuracy under any forwarding mode.

X. TARGETED LITERATURE AUDIT AND FEATURE-RECIPE EVALUATIONS

A. Targeted Audit of SDN-IDS Literature

To assess how prevalent these evaluation flaws are in the SDN-IDS literature, we conducted a targeted audit of 12 representative papers (eleven focusing on SDN or InSDN, and one non-SDN control). As documented in Table XII, ten papers were read in full text.

Our audit finds recurring methodological gaps:

1) **Absence of Host Grouping:** Among all InSDN papers read in full, none reports a host-grouped or capture-grouped validation split. Most employ simple randomized partitions (such as `train_test_split`) or fail to document their split mechanism entirely.
2) **Use of Network Identifiers as Model Inputs:** Three papers explicitly include non-generalizable network identifiers (Flow IDs, IP addresses, or port numbers) among their feature sets. Notably, in SDNGuardStack [5] the top-15 mutual-information features include Flow ID (ranked first), Src IP, Dst IP, Src Port and Dst Port, and the authors' SHAP ranking puts Flow ID first. OptiXGB-IDS [8] and Mehmood et al. [10] retain source or destination port as inputs; in the former the surrogate tree's major split is Dst Port, and in the latter Dst Port is the strongest negative SHAP contributor.
3) **Offline Evaluation of Controller Claims:** Multiple papers frame their models as lightweight controller-side defenses, yet evaluate exclusively on offline CSVs without deploying within an SDN controller.

B. Feature-Recipe Evaluations on InSDN

To determine whether the evaluation gap is specific to feature selection, we implemented four generic feature recipes approximating choices in the literature, evaluated with GPU XGBoost across 10 seeds (Table XI):

1) **Numeric features only (77 features):** Random split achieves 0.9998 BAcc (0.9991 8-class); held-out source hosts drops to 0.943 BAcc (0.788 8-class).
2) **Numeric + ports (79 features):** Random split achieves 0.9998 BAcc (0.9998 8-class); held-out source hosts drops to 0.856 BAcc (0.687 8-class).
3) **Numeric + identifiers (83 features):** Random split achieves 1.0000 BAcc (0.9999 8-class); held-out source hosts drops to 0.941 BAcc (0.804 8-class).
4) **Top-15 by Mutual Information:** Selected on full data prior to splitting. Mutual information automatically selects Flow ID, Src IP and Dst IP. Random split achieves 1.0000 BAcc (0.9999 8-class); held-out hosts drops to 0.941 BAcc (0.835 8-class, widest CI 0.241).

Across all recipes, random splits achieve near-perfect scores ($\geq$ 0.9998), but performance degrades under host-grouped evaluation. Under host grouping, adding identifier columns yields 0.941, comparable to numeric features alone (0.943), which is consistent with decision trees relying on numeric features when test-set IPs are unseen. However, under random splits, computing mutual information across the unpartitioned dataset selects Flow ID, Src IP and Dst IP among the top features, enabling near-perfect scores (1.0000) by memorizing training-set identifiers across partitions.

XI. METHODOLOGICAL GUIDELINES AND EVALUATION CHECKLIST

To improve rigor in SDN intrusion detection research, we propose seven concrete methodological guidelines:

TABLE XV
FORWARDING APPLICATION VS. WHAT THE CONTROLLER OBSERVES: BENIGN TRAFFIC FROM THE SAME GENERATOR AND SEEDS (5 HOSTS → 1 HTTP SERVER, PLUS PINGS), 1-SWITCH MININET/RYU TESTBED, ∼99 S CAPTURE PER RUN, 10 RUNS PER MODE, MEAN±95% CI. ENTRIES = FLOW ENTRIES RETURNED PER FLOW-STATISTICS REPLY (1 S POLLING).

Mode	TCP PacketIns	All PacketIns	FlowMods	MAC-pair entries	IP-specific entries
MAC-pair (as ONOS in InSDN)	18.0±5.0	267.8±18.2	33.9±3.8	7.0±0.0	0.0±0.0
5-tuple, 10 s idle	10881.3±793.8	11159.9±795.4	10928.3±791.8	0.0±0.0	112.4±2.3
5-tuple, 2 s idle	10675.5±803.9	11275.8±814.7	11040.4±805.9	0.0±0.0	27.0±0.4

1) **Verify Controller Observability:** Researchers proposing controller-side defenses must verify that input features correspond to OpenFlow control-plane primitives. If features require full-packet timing moments, the architectural cost of data-plane polling or flow-table modifications must be explicitly accounted for.
2) **Report Host-Disjoint Splits:** Random flow splits should not be the sole basis for claims of generalizable detection. Report a host-disjoint evaluation alongside, so that source hosts in the test set are withheld from training ($\mathcal{H}_{\text{train}} \cap \mathcal{H}_{\text{test}} = \emptyset$), and state how many dominant hosts each split separates.
3) **Purge Non-Generalizable Network Identifiers:** All non-statistical network metadata (such as Flow IDs, IP addresses, MAC addresses, and ephemeral transport ports) must be removed prior to feature extraction.
4) **Audit Dataset Host Concentration:** Authors must report the distribution of flows per host. In datasets where a handful of hosts dominate total volume, Leave-One-Host-Out (LOHO) testing should be reported alongside aggregate metrics.
5) **Report Redundancy Audits:** Report the share of test rows that duplicate training rows, and results with and without cross-split duplicates removed. In our data, removal left random-split scores unchanged and moved host-grouped scores in both directions, so both should be shown.
6) **Evaluate Forwarding Application Sensitivity:** Detectors relying on control-plane metrics (such as PacketIn rates) must evaluate sensitivity to forwarding application policies, entry granularity, and idle timeout configurations.
7) **Report Session-Identity Baselines and Label Coverage:** When benign and attack traffic are recorded in separate sessions, report trivial identity rules (an any-IP rule and a host-memory rule learned on training data), the attack windows per deployment and class, and, where possible, record benign and attack traffic interleaved.

XII. DISCUSSION AND LIMITATIONS

A. Scope and Experimental Boundaries

The controller-side results concern one benchmark (InSDN), one controller (ONOS with its default reactive forwarding) and two deployments (Metasploitable-2 and OVS). CICIDS2017 is a non-SDN reference used only for the split analysis. The split analysis uses one classifier family (XGBoost) on flow features, and other architectures may respond differently. We attempted a third candidate, UKMDDoSDN [36], but its public PCAPs contain no benign traffic and no OpenFlow control channel (Appendix A). We frame this work as a measurement study and methodological critique, not as a deployable end-to-end detector.

B. Threats to Validity

Construct validity. Window labels come from matching PacketIn 5-tuples to CSV records that were produced from separate end-host recordings; the alignment of the two recordings is unverified. Labelled attack windows are those in which a CSV-matched flow crossed the controller, which may favour easy windows, and OVS DDoS and OVS brute-force have no labelled attack windows (Table II).

Internal validity. Seed intervals reflect initialization and fold assignment, not the sampling of captures (the capture-level bootstrap addresses the latter). The host-held-out results rest on ten correlated splits in which eight hosts hold 63% of the rows, so the host logging shows an association, and leave-one-host-out is a single case per host. Our flow reassembler follows CICFlowMeter's default expiration rules but is not CICFlowMeter, and part of the mismatch with the CSV may stem from that difference. The host-identity rules are simple, so their score is a lower bound on what identity can explain.

External validity. One dataset, one controller and one forwarding application were studied. The Mininet/Ryu study is benign-only with at most three software switches; proactive rule installation, hardware TCAM limits, multiple controllers and production traffic are not covered.

C. Ground-Truth Labeling Trade-Offs

Strict 5-tuple matching grounds controller windows in benchmark labels but discards unlabelled traffic in attack sessions. Under an alternative broader rule that labels any window containing IP addresses observed during attacks, overall balanced accuracy is lower (MLP 0.881 at 1 s, 0.887 at 5 s), and transfer from OVS to Metasploitable-2 is 0.977 against 0.924 in the reverse direction.

D. Deployment Feasibility

An operational controller-side detector would inspect the control channel through port mirroring or controller event handlers. In our testbed, forwarding granularity and idle timeout change PacketIn volume by two to three orders of magnitude (Section IX). Detectors built on PacketIn rates may therefore need recalibration when the forwarding application changes; we did not evaluate any detector under a different forwarding mode, so this is a hypothesis and not a result.

XIII. Conclusion

We audited what an SDN controller observes in InSDN and how that benchmark is evaluated. In the 44 ONOS captures, no flow-table entry or FlowMod matches IP addresses or ports, so per-flow counters are absent, while an IPv4 punt rule delivers packet headers to the controller; software reassembly of flows from those PacketIns is partial. A controller-side MLP reaches 0.946 balanced accuracy on unseen captures, but because benign and attack traffic were recorded in separate sessions, trivial rules based on IP presence or on host identity explain much of the separation, and the score cannot be read as attack detection. Under strict labelling, whole deployment–class cells (OVS DDoS, OVS brute-force) carry no attack windows and probing dominates the rest.

On flow features, random splits give ≥ 0.9917 balanced accuracy on both InSDN and CICIDS2017, whereas holding out source hosts gives 0.907 ± 0.085 and 0.641 ± 0.056; the InSDN dispersion is consistent with which of eight dominant hosts is withheld, and leave-one-host-out recall on the primary attacker is 51.6% (DoS 34.7%). None of the 12 audited SDN-IDS papers reports a host-grouped split. These results come from one dataset and one controller, and the split analysis confirms known leakage mechanisms on an SDN benchmark rather than introducing a new one. We offer seven evaluation guidelines and release the code and result files so that the measurements can be repeated and challenged.

Appendix A
Candidate Second Datasets Examined

In addition to InSDN and CICIDS2017, we examined two candidate datasets for control-channel analysis. For UKMDDoSDN [36], we fetched `syn_flood_combined.pcap` (18 922 240 B; 199 273 packets) and `udp_flood_combined.pcap` (15 420 748 B; 110 518 packets). Parsing confirmed that neither capture contains packets on TCP port 6633 or 6653; they represent data-plane host captures without OpenFlow control traffic, and contain no benign class. For LAN-SDN-NIDS [37], its record describes flow-level records with 11 control-plane metrics, but raw OpenFlow control-plane captures are not available. Neither dataset could support control-channel reassembly.

Data and Code Availability

InSDN CSVs and PCAPs are publicly available (sources and SHA-256 digests in `data/MANIFEST.md`). The OpenFlow 1.3 parser, feature extraction, evaluation code, the per-experiment result files and the table and figure generators are in the project repository (https://github.com/adhyan-jain/Proteus, directory `research/insdn_controller`). The script `reproduce.sh` is provided to regenerate the result files from the raw data (it requires a CUDA GPU), and `paper/verify_numbers.py` checks every number quoted in the text against the stored result files. Experiments ran in Python 3.12 with PyTorch 2.11 (CUDA 12.8) and XGBoost 3.4 on one NVIDIA RTX 4060 laptop GPU. Capture hashes are in `capture_hashes.sha256`.

Acknowledgment

All three authors contributed equally to the study design, empirical analysis, and manuscript preparation. An AI assistant (Claude, Anthropic) was utilized to assist with scripting, literature synthesis, and drafting; the authors independently verified all experimental results against stored result files and assume full responsibility for the manuscript.

References

[1] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[2] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[3] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, "InSDN: A novel SDN intrusion dataset," *IEEE Access*, vol. 8, pp. 165 263–165 284, 2020.

[4] M. S. Ataa, E. E. Sanad, and R. A. El-khoribi, "Intrusion detection in software defined network using deep learning approaches," *Scientific Reports*, vol. 14, p. 29159, 2024.

[5] Ashikuzzaman, M. S. Abhi, M. Rahman, M. M. Ahmed, M. M. Hasan, and M. A. Arif, "SDNGuardStack: An explainable ensemble learning framework for high-accuracy intrusion detection in software-defined networks," arXiv preprint arXiv:2604.20934, 2026.

[6] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[7] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2017, pp. 253–262.

[8] M. T. H. Saykat, M. E. Haque, F. Al Farid, R. Hossen, J. Uddin, and H. A. Karim, "An efficient and interpretable intrusion detection framework for software-defined networks with multi-class imbalanced data using genetic and GAN-based optimization," *Scientific Reports*, vol. 16, 2026.

[9] M. Mujahid, A. R. Mirdad, F. S. Alamri, A. Ara, and A. Khan, "Software defined network intrusion system to detect malicious attacks in computer Internet of Things security using deep extractor supervised random forest technique," *PeerJ Computer Science*, vol. 11, p. e3103, 2025.

[10] S. Mehmood, R. Amin, J. Mustafa, M. Hussain, F. S. Alsubaei, and M. D. Zakaria, "Distributed denial of services (DDoS) attack detection in SDN using optimizer-equipped CNN-MLP," *PLOS ONE*, vol. 20, no. 1, p. e0312425, 2025.

[11] Open Networking Foundation, "OpenFlow switch specification, version 1.3.0," ONF TS-006, 2012.

[12] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2010, pp. 305–316.

[13] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[14] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[15] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. 9th IEEE European Symposium on Security and Privacy (EuroS&P)*, 2024, pp. 658–675.

[16] U. S. Roy and M. J. Minu, "Unmasking shortcut learning in IoT intrusion detection: A forensic, multi-paradigm evaluation of feature dependence and data leakage," arXiv preprint arXiv:2609.28725, 2026.

[17] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022.

[18] D. Pinto, J. Vitorino, E. Maia, I. Amorim, and I. Praça, "Flow exporter impact on intelligent intrusion detection systems," in *Proc. 11th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2025, pp. 289–298.

[19] A. Pekar and R. Jozsa, "Early-stage anomaly detection: A study of model performance on complete vs. partial flows," arXiv preprint arXiv:2407.02856, 2024.

[20] ——, "Evaluating ML-based anomaly detection across datasets of varied integrity: A case study," *Computer Networks*, vol. 251, p. 110617, 2024.

[21] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and E. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Proc. Int. Conf. Risks and Security of Internet and Systems (CRiSIS)*, 2022, pp. 18–33.

[22] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE Conf. Communications and Network Security (CNS)*, 2022, pp. 280–288.

[23] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS flooding attack detection using NOX/OpenFlow," in *Proc. IEEE Local Computer Network Conference (LCN)*, 2010, pp. 408–415.

[24] S. M. Mousavi and M. St-Hilaire, "Early detection of DDoS attacks against SDN controllers," in *Proc. Int. Conf. Computing, Networking and Communications (ICNC)*, 2015, pp. 77–81.

[25] S. Shin, V. Yegneswaran, P. Porras, and G. Gu, "AVANT-GUARD: Scalable and vigilant switch flow management in software-defined networks," in *Proc. ACM Conf. Computer and Communications Security (CCS)*, 2013, pp. 413–424.

[26] H. Wang, L. Xu, and G. Gu, "FloodGuard: A DoS attack prevention extension in software-defined networks," in *Proc. 45th Annual IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2015, pp. 239–250.

[27] M. Dhawan, R. Poddar, K. Mahajan, and V. Mann, "SPHINX: Detecting security attacks in software-defined networks," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2015.

[28] P. Berde, M. Gerola, J. Hart, Y. Higuchi, M. Kobayashi, T. Koide, B. Lantz, B. O'Connor, P. Radoslavov, W. Snow, and G. Parulkar, "ONOS: Towards an open, distributed SDN OS," in *Proc. 3rd Workshop on Hot Topics in Software Defined Networking (HotSDN)*, 2014, pp. 1–6.

[29] M. S. Towhid and N. Shahriar, "Early detection of intrusion in SDN," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2023, pp. 1–6.

[30] M. Abdallah, N.-A. Le-Khac, H. Jahromi, and A. D. Jurcut, "A hybrid CNN-LSTM based approach for anomaly detection systems in SDNs," in *Proc. 16th Int. Conf. Availability, Reliability and Security (ARES)*, 2021, pp. 34:1–34:7.

[31] R. A. Elsayed, R. A. Hamada, M. I. Abdalla, and S. A. Elsaid, "Securing IoT and SDN systems using deep-learning based automatic intrusion detection," *Ain Shams Engineering Journal*, vol. 14, no. 10, p. 102211, 2023.

[32] A. Alqhatani, S. Mehmood, R. Amin, M. S. Alshehri, A. H. Alshehri, and F. Asiri, "Deep memory for deep threats: A novel architecture combining GRUs and deep learning models for IDS," *PLOS ONE*, vol. 20, no. 10, p. e0332752, 2025.

[33] H. R. Malik, M. Salman, Z. Ahmed, and N. Maryam, "Improving robustness and interpretability of deep learning-based intrusion detection in software-defined networks," Unreviewed manuscript, https://muhammadsalman1205.vercel.app/research-paper-icit2026.pdf, 2026.

[34] S. Rohith, G. Logeswari, K. Tamilarasi, and G. Sudhakaran, "A federated deep learning approach for SDN security with quantum optimized feature selection and hybrid MSDC net architecture," *Scientific Reports*, vol. 16, p. 8038, 2026.

[35] M. T. Saleh and A. H. Hamad, "SICA: Generation and deep learning-based evaluation of a novel dataset for intrusion detection in SDN-IoT environments," *Journal Européen des Systèmes Automatisés (JESA)*, vol. 59, no. 1, pp. 69–77, 2026.

[36] Universiti Kebangsaan Malaysia, "UKMDDoSDN: A DDoS attacks dataset for software-defined networks, version 2," Mendeley Data, https://doi.org/10.17632/43nfv26gpk.2, 2025.

[37] J. Arévalo-Herrera, J. E. Camargo, J. I. Martínez Torre, J. M. Ramírez, and T. Zona-Ortiz, "A multi-class SDN intrusion detection dataset with synchronized OpenFlow control-plane telemetry," *Data*, vol. 11, no. 8, p. 195, 2026.